REDUCED LEVEL CALCULATOR REPORT: Group: SEVEN (7)

NAME	REG. NUMBER
OBANGATEK EMMANUEL	25/1/324/D/001
NATUMANYA CYNTHIA ARINDA	25/1/324/D/334
NAKAFEERO CYNTHIA	25/1/324/D/1145
KIZITO SIMON PETER	25/1/324/D/303

A Python Application for Rise and Fall and Height of Collimation Levelling.

Introduction and Problem Identification

Levelling is the standard field method for finding the heights of points in land and engineering surveying. After the fieldwork, the surveyor must "reduce" the level book: convert the staff readings (backsight BS, intermediate sight IS, foresight FS) into reduced levels (RL). Two methods are used, Rise and Fall and Height of Collimation (HoC). These were our concentration.

This causes real problems which include;

- The arithmetic is repetitive,
- Change points (CP) are easy to mishandle.
- The misclosure must be shared over the instrument setups, and we survey students and surveyors often apply it per point instead of per setup.
- The arithmetic checks are skipped or done too late to be useful.

Surveying students and practicing surveyors need a fast, reliable and repeatable way to reduce level-book data by both methods, adjust it, check it, and present it in standard tables. This is what we addressed through developing a reduced level calculator.

Objectives (for creating the program)

To accept the initial RL of the first benchmark and the final RL of the last benchmark, and a complete list of BS, IS and FS readings, pasted in or imported from a file.

To calculate reduced levels by the Rise and Fall method and the Height of Collimation method.

To apply corrections by dividing the misclosure over the setups and distributing it appropriately.

To display the results in properly arranged surveying tables and identify the change points, check accuracy with the standard arithmetic checks and display the difference between the two methods' results.
To export the tables to CSV files that open in Microsoft Excel.
To run as a desktop application with a graphical interface, so readings are not typed one by one into a console.

3. Proposed Solution

We built the **Reduced Level Calculator**, a Python desktop program. The user enters two benchmark RLs and pastes (or imports from .csv, .txt or .xlsx) the BS, IS and FS columns. One click on **Calculate** produces:

- a Rise and Fall table and a Height of Collimation table, each with initial RL, adjustment, adjusted RL and remarks (TBM, A, B, C (CP) ...);
- a Difference table comparing the two methods;
- a Checks tab showing the arithmetic checks, misclosure, number of setups and correction per setup.

The **Export CSV** button saves the three tables for Excel. The program removes manual arithmetic, applies the same rules every time, and flags faulty data with a clear message.

Methodology

Tools

Python 3.14 in PyCharm; Tkinter for the interface; the csv module for export; openpyxl (optional) for reading .xlsx; unittest for testing; Git and GitHub for version control and deployment.

Program design

The program is split into modules so the calculation code could be tested without opening a window.

Module	Responsibility
levelling_core.py	Parsing, validation, both methods, corrections, remarks, checks, table building
file_io.py	CSV export; import from .csv, .txt, .xlsx
gui.py	Tkinter window, buttons, tabbed tables
main.py	Starts the application
test_levelling.py	Nine automated tests

Algorithms

Input rules. The first row has a BS only; the last row has an FS only; an intermediate point has an IS only; a change point has an FS and the new BS on the same row.

Rise and Fall. For each point, d = (previous reading) − (current reading). A positive d is a rise and a negative d is a fall. RL(new) = RL(previous) + d. After a change point, the next comparison starts from the new BS.

Height of Collimation. HI = RL + BS. For each point, RL = HI − (IS or FS). At a change point a new HI is computed from that point's RL and the new BS.

Corrections. Misclosure e = known final RL − computed last RL. The number of setups n equals the number of BS readings. Correction per setup c = e / n. A point observed in setup k receives k × c, and a change point belongs to the setup it closes. Adjusted RL = RL + correction. The first benchmark receives no correction.

Arithmetic checks. Sum BS − Sum FS = Sum Rise − Sum Fall = Last RL − First RL.

Method comparison. Difference = adjusted RL (Rise and Fall) − adjusted RL (HoC), which should be 0.000 at every point.

4.4 Inputs and outputs

Inputs: initial RL, final RL and the BS | IS | FS readings (pasted tab or comma separated, or imported from a file; an imported file may also contain Initial RL and Final RL lines). **Outputs:** the on-screen tables, the checks summary and three CSV files.

Testing and Results

Test 1: field data from our Excel level book

Inputs: initial RL 49.873, final RL 48.710, seven readings with two change points.

Point	Initial RL	Adjustment	Adjusted RL
TBM	49.873	0.000	49.873
A	49.559	0.002	49.561
B	49.739	0.002	49.741
C (CP)	50.568	0.002	50.570
D	51.074	0.004	51.078
E (CP)	50.776	0.004	50.780
TBM	48.704	0.006	48.710

The program reproduced every value of the Excel table. The misclosure was 0.006 m over 3 setups, giving 0.002 per setup. The checks gave Sum BS = 6.962, Sum FS = 8.131, difference −1.169; Sum Rise = 1.515, Sum Fall = 2.684, difference −1.169; last RL − first RL = −1.169. Both checks passed, and the Difference table was 0.000 at every point.

Test 2: different data

With initial RL 100 and final RL 98.5 and the same readings, the misclosure was −0.331 m. The corrections (−0.110, −0.221, −0.331) were applied correctly, and the last adjusted RL equalled 98.500.

Test 3: import and validation

Import was tested with a CSV (with header and RL lines), a tab-separated text file and an Excel file. Invalid data was rejected with clear messages: a first row without a BS, a last row that is not an FS, text inside a reading, and a middle row mixing IS with BS or FS.

Test 4: automated tests

Nine unit tests (Rise and Fall values, HoC agreement, corrections, full calculation, validation, point naming, file import) all passed.

Challenges Encountered

SyntaxError: invalid syntax (RLcalc.py)

Exception in Tkinter callback when clicking Calculate. A faulty copy of one function (result was not a list); the error appeared only in the console

Confusion between warnings and errors in PyCharm. We got yellow warnings which threw us into confusion since we did not know that it was as a result of

Imported files failing

Tiny rounding differences

Limitations and Future Work

- The program assumes one line of levels between two benchmarks (open at both ends with known RLs); closed loops and multiple lines are not yet handled.
- Misclosure is shared equally per setup; weighting by distance could be added.
- An allowable-misclosure check (c√K), an .xlsx export, a longitudinal profile plot, and a decimal-comma option for other regional settings.

Conclusion

The Reduced Level Calculator meets all ten objectives. It reduces level-book data by both methods, applies corrections per instrument setup, identifies change points, performs the arithmetic checks, compares the two methods and exports to Excel. It reproduced our hand-checked field data exactly and passed all automated tests. The main lessons were to separate logic from the interface, to test early, and to use version control instead of manual copies.